TP 1 – Python, simulation et premières variables aléatoires

Yiye Jiang

Année universitaire 2026–2027

Objectifs. Ce premier TP poursuit deux objectifs : revoir les outils Python qui seront utilisés tout au long du module, puis les appliquer aux premières notions du cours.

Environnement de travail. Deux environnements sont proposés, au choix : **Spyder** (éditeur de scripts `.py`, console interactive et explorateur de variables) ou **Jupyter Notebook** (fichier `.ipynb` réunissant le code et les résultats). Les exercices sont les mêmes dans les deux environnements. Nous utiliserons NumPy, Matplotlib et SciPy. Sauf indication contraire, privilégier les opérations vectorisées aux boucles Python. Les instructions d'installation et d'utilisation de Spyder et Jupyter sous Windows, macOS et Linux figurent en annexe.

Pour la séance évaluée : le **rapport final** devra être réalisé dans **Jupyter Notebook**, contenir le code Python, les sorties et les graphiques, et aussi les **commentaires des résultats**, puis être **exporté au format HTML ou PDF**. Spyder peut être utilisé pour travailler, mais un simple script `.py` ne constitue pas le rapport à rendre.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(2026)
```

1. Remise à niveau Python orientée proba-stat

Exercice 1 – Tableaux, tri, filtrage et indices

On considère les données suivantes :

```
x = np.array([12.4, 10.1, 9.8, 15.2, 11.7,
              13.3, 8.9, 12.4, 14.1, 10.7])
```

1. Afficher le nombre d'éléments de `x`, son type, ses trois premières valeurs, ses trois dernières valeurs, puis une valeur sur deux.
2. Trier les valeurs de `x` par ordre croissant avec `np.sort`, puis utiliser `np.argsort` pour obtenir les indices correspondant à cet ordre.
3. $$10 \leq x_i \leq 13.$$

   Afficher les valeurs satisfaisant cette condition, puis leurs indices dans le tableau initial.
4. Trouver le nombre de valeurs satisfont la condition précédente à l'aide d'une commande Python ?
5. Déterminer la plus grande valeur de `x` et l'indice de sa position. Procéder de même pour la plus petite valeur.
6. Calculer la moyenne, la médiane, l'écart-type et les quantiles d'ordres 0.25, 0.50 et 0.75.
7. Comparer les résultats de `np.std(x)` et de `np.std(x, ddof=1)`.

Aide-mémoire Python :

```
x[condition]                # filtrer des valeurs
np.where(condition)         # indices satisfaisant une condition
np.sort(x)                  # valeurs triees
np.argsort(x)               # indices du tri
np.argmax(x), np.argmin(x)
```

```
np.mean(x), np.median(x)
np.var(x), np.std(x)
np.quantile(x, q)
```

Exercice 2 – Matrices et opérations utiles pour la suite du module

On considère

```
t = np.array([0., 1., 2., 3., 4.])
y = np.array([2.1, 1.7, 1.1, 0.6, 0.2])
X = np.column_stack((np.ones(len(t)), t))
```

1. Afficher la matrice `X`, ses dimensions et sa transposée `X.T`.
2. Calculer $X^\top X$ et $X^\top y$ à l'aide de l'opérateur de multiplication matricielle `@`.
3. On considère les deux matrices suivantes :

   ```
   A = np.array([[1., 2.], [3., 4.]])
   B = np.array([[2., 0.], [1., 2.]])
   ```

   Calculer `A * B`, `np.multiply(A, B)` et `A @ B`, puis comparer les résultats.
4. Résoudre le système

   $$(X^\top X)b = X^\top y$$

   à l'aide de `np.linalg.solve`.
5. On définit une grille de paramètres et une fonction numérique :

   ```
   theta = np.linspace(0.01, 0.99, 999)
   ell = 37*np.log(theta) + 63*np.log(1-theta)
   ```

   Déterminer l'indice du maximum de `ell`, puis la valeur correspondante de `theta`.

   Visualisation avec Matplotlib.

   a) Tracer `ell` en fonction de `theta` à l'aide de `plt.plot`.

   b) Définir une deuxième fonction sur la même grille :

   ```
   ell2 = 50*np.log(theta) + 50*np.log(1-theta)
   ```

   Superposer les courbes de `ell` et de `ell2`. Ajouter une légende (`label`, `plt.legend`), un titre (`plt.title`) et les intitulés des axes (`plt.xlabel`, `plt.ylabel`).

2. Une variable aléatoire comme fonction

Exercice 3 – Deux dés : lois de I, Z et du couple (I, Z)

On lance deux dés équilibrés et indépendants. Une issue est représentée par $(i, j) \in \Omega = \{1, \dots, 6\}^2$. Les 36 issues sont équiprobables.

```
omega = np.array([(i, j) for i in range(1, 7)
                         for j in range(1, 7)])
```

On définit les variables aléatoires

$$I(i,j) = i, \qquad Z(i,j) = \mathbf{1}_{\{i=j\}} = \begin{cases} 1 & \text{si les deux dés donnent le même résultat,} \\ 0 & \text{sinon,} \end{cases}$$

ainsi que le couple (I, Z).

1. Construire en Python les tableaux contenant les valeurs de I et de Z pour chacune des 36 issues.
2. Déterminer les ensembles (donc les supports) $I(\Omega)$ et $Z(\Omega)$ à l'aide de `np.unique`.
3. Pour I et pour Z, utiliser `np.unique(..., return_counts=True)` afin de construire leurs fonctions de masse. Vérifier numériquement que, dans chaque cas, la somme des probabilités vaut 1.
4. Représenter les fonctions de masse de I et de Z à l'aide de `plt.stem`.

5. Construire la **fonction de masse jointe** du couple (I, Z) :

$$p_{I,Z}(a, z) = \mathbb{P}(I = a,\, Z = z).$$

La représenter par un tableau de taille 6×2, dont les lignes correspondent à $a = 1, \ldots, 6$ et les colonnes à $z = 0, 1$:

	$z = 0$	$z = 1$
$a = 1$	$p_{I,Z}(1,0)$	$p_{I,Z}(1,1)$
$a = 2$	$p_{I,Z}(2,0)$	$p_{I,Z}(2,1)$
$\vdots$	$\vdots$	$\vdots$
$a = 6$	$p_{I,Z}(6,0)$	$p_{I,Z}(6,1)$

Initialiser ce tableau avec `np.zeros((6, 2))`, puis calculer ses cases. Vérifier que la somme des 12 probabilités est égale à 1.

6. Représenter la loi jointe sous forme de carte de chaleur (*heatmap*) à l'aide de `plt.imshow` et `plt.colorbar`.
7. À partir du tableau de la loi jointe, retrouver $p_I(a)$ en sommant les termes de chaque **ligne**, puis $p_Z(z)$ en sommant ceux de chaque **colonne**. Il s'agit des **lois marginales** du couple. Utiliser `np.sum(..., axis=1)` et `np.sum(..., axis=0)`. Vérifier que chacune des deux fonctions de masse a une somme égale à 1.
8. Vérifier, pour tous $a \in I(\Omega)$ et $z \in Z(\Omega)$, si

$$p_{I,Z}(a, z) = p_I(a)\, p_Z(z).$$

En déduire si les variables aléatoires I et Z sont **indépendantes**.

3. Lois discrètes usuelles et fonctions de masse

Aide-mémoire – Les lois usuelles dans `scipy.stats`

Le module `scipy.stats` permet de simuler des variables aléatoires et d'évaluer les fonctions de masse, les densités et les fonctions de répartition de nombreuses lois. Les tableaux ci-dessous constituent un **aide-mémoire** : il n'est pas nécessaire de mémoriser toutes ces lois pour ce TP.

```
from scipy import stats
# Exemple : stats.binom.pmf(2, n=20, p=0.05)
# Exemple : stats.expon.pdf(3, scale=2)
```

Principales méthodes de `scipy.stats`.

Méthode	Rôle
`rvs(...)`	Simuler une ou plusieurs réalisations.
`pmf(k, ...)`	Évaluer la fonction de masse en k (loi discrète).
`pdf(x, ...)`	Évaluer la densité en x (loi à densité).
`cdf(x, ...)`	Évaluer la fonction de répartition en x.
`ppf(q, ...)`	Obtenir le quantile d'ordre q.

On utilise ces méthodes avec la syntaxe `stats.nom_loi.methode(...)`. Par exemple, on peut écrire `stats.norm.rvs(...)`. Pour obtenir de l'aide : `help(stats.expon)` ou `stats.expon?` dans une console compatible.

Quelques lois disponibles dans `scipy.stats`.

Commande SciPy	Loi correspondante
`norm()`	Normale $\mathcal{N}(0,1)$.
`gamma(a=...)`	Gamma $\Gamma(a,1)$.
`beta(a=..., b=...)`	Bêta $\beta(a,b)$.
`t(df=n)`	Student à n degrés de liberté.
`cauchy()`	Cauchy standard.
`expon(scale=1/lambda)`	Exponentielle $\mathcal{E}(\lambda)$.
`uniform(loc=..., scale=...)`	Uniforme sur $[\text{loc}, \text{loc}+\text{scale}]$.
`randint(low, high)`	Uniforme sur $\{\text{low}, \ldots, \text{high}-1\}$.
`binom(n=..., p=...)`	Binomiale $\mathcal{B}(n,p)$.
`geom(p=...)`	Géométrique $\mathcal{G}(p)$.
`poisson(mu=...)`	Poisson $\mathcal{P}(\mu)$.
`chi2(df=...)`	Khi-deux $\chi^2(\text{df})$.

Exercice 4 – Choisir une loi et utiliser sa fonction de masse

Pour chacune des situations suivantes, définir une variable aléatoire, préciser l'ensemble de ses valeurs possibles et identifier sa loi usuelle :

a) Une pièce est défectueuse avec probabilité p. On code le résultat par 1 si la pièce est défectueuse, et par 0 sinon.

b) On contrôle indépendamment 20 pièces, chacune ayant une probabilité 0.05 d'être défectueuse, et l'on compte les pièces défectueuses.

c) On répète des essais indépendants jusqu'au premier succès et l'on note le rang de ce succès.

Pour la suite, $X \sim \mathcal{B}(20, 0.05)$ pour modéliser le nombre des pièces défectueuses.

1. Calculer la fonction de masse de X avec `stats.binom.pmf`, puis la représenter avec `plt.stem`.
2. À partir de cette fonction de masse, calculer, par sommation ou en utilisant l'événement contraire :

$$\mathbb{P}(X = 0), \qquad \mathbb{P}(X \leq 2), \qquad \mathbb{P}(X \geq 1).$$

3. Simuler 10 000 réalisations de X avec `rng.binomial` ou `stats.binom.rvs`. Sur un même graphique, comparer les fréquences empiriques à la fonction de masse théorique.

Rappel SciPy :

```
stats.binom.pmf(k, n=20, p=0.05)
stats.binom.rvs(n=20, p=0.05, size=10000)
```

Déduire la loi d'une nouvelle variable aléatoire. On définit $V = \min(X, 2)$: si le nombre de pièces défectueuses est inférieur à 2, on conserve sa valeur exacte ; sinon, on retient la valeur 2.

1. Déterminer les valeurs possibles de V.
2. Déduire la **fonction de masse de** V de celle de X.
3. Appliquer `np.minimum` aux réalisations simulées de X, puis comparer les fréquences obtenues à la loi théorique de V.

5. Boîte à outils à conserver pour les prochains TP

Les commandes suivantes seront réutilisées au cours du module :

Commande	**Utilisation typique**
`x[condition]`	filtrer un échantillon
`np.where`, `np.argmax`, `np.argmin`	localiser des observations ou un optimum
`np.sort`, `np.argsort`	trier les valeurs ou récupérer l'ordre des indices
`np.mean`, `np.var`, `np.std`	statistiques empiriques
`np.quantile`	quantiles empiriques et intervalles
`A @ B`, `A.T`, `np.linalg.solve`	calcul matriciel, régression
`A * B`, `np.multiply`	produit terme à terme (tableaux)
`np.log`, `np.sum`	log-vraisemblance
`np.linspace + np.argmax`	recherche d'un maximum sur une grille
`stats.*.pmf / stats.*.pdf`	fonctions de masse et densités
`stats.*.cdf / stats.*.ppf`	fonctions de répartition, quantiles
`rng.*`	simulation de variables aléatoires
`plt.plot`, `plt.legend`	tracés et légendes
`plt.xlabel`, `plt.ylabel`	intitulés des axes
`plt.axhline`, `plt.axvline`	droites de référence

Documentation utile : NumPy, SciPy (module `stats`) et Matplotlib.

Annexe – Travailler avec Spyder ou Jupyter

Installer, si possible, l'environnement choisi avant la séance. Les deux solutions ci-dessous permettent d'effectuer les mêmes calculs en Python.

Option A – Spyder : éditeur et console interactifs

Installation (Windows, macOS, Linux). Télécharger **Spyder** sur spyder-ide.org/download et choisir l'installateur correspondant à son système :

— **Windows :** exécuter l'installateur téléchargé, puis ouvrir Spyder depuis le menu Démarrer.
— **macOS :** télécharger la version adaptée au processeur du Mac (Apple Silicon ou Intel), installer Spyder, puis le lancer depuis le dossier Applications.
— **Linux :** sur le site officiel de Spyder, cliquer sur *Download for Linux* pour télécharger le fichier `.sh`. Ouvrir un terminal et exécuter :

```
cd ~/Downloads
bash Spyder-Linux-x86_64.sh
```

Si le dossier porte le nom « Téléchargements », utiliser `cd ~/Téléchargements`. Adapter le nom du fichier `.sh` au fichier effectivement téléchargé. Suivre les instructions affichées, puis lancer Spyder depuis le menu des applications. Avec `bash`, ni `sudo` ni `chmod` ne sont nécessaires.

Les installateurs autonomes de Spyder fournissent un environnement Python comprenant notamment NumPy, SciPy et Matplotlib. **Il n'est pas nécessaire d'installer Jupyter pour travailler dans Spyder.**

Pendant le TP. Créer un script `tp1.py`. Dans l'éditeur, les différentes parties peuvent être séparées par `# %%` :

```
# %% Exercice 1
import numpy as np
x = np.array([3, 1, 2])
print(np.sort(x))

# %% Exercice 2
print(np.mean(x))
```

Exécuter : `F9` (ligne ou sélection), `Ctrl+Entrée` / `Cmd+Entrée` (cellule), `F5` (script entier). Les résultats s'affichent dans la console IPython, les données dans l'explorateur de variables et les graphiques dans le volet Graphiques (selon les réglages).

Option B – Jupyter Notebook : code, figures et résultats réunis

1. Installer Python 3 (si nécessaire).

— **Windows :** python.org/downloads/windows/, puis vérifier `py -3 -version` dans PowerShell.
— **macOS :** python.org/downloads/macos/, puis vérifier `python3 -version` dans Terminal.
— **Linux :** vérifier `python3 -version`. Pour Ubuntu/Debian, si nécessaire :

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

2. Installer les bibliothèques et Jupyter dans un dossier de TP.

Windows (PowerShell)

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install numpy scipy matplotlib notebook nbconvert
.\.venv\Scripts\python.exe -m notebook
```

macOS / Linux (Terminal)

```
python3 -m venv .venv
./.venv/bin/python -m pip install numpy scipy matplotlib notebook nbconvert
./.venv/bin/python -m notebook
```

Créer un notebook `tp1.ipynb` et exécuter les cellules avec `Maj+Entrée`. Les résultats et les graphiques s'affichent sous les cellules.

Vérification commune

Dans l'environnement choisi, exécuter :

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
print(np.__version__)
```

Séance notée : rapport obligatoire produit avec Jupyter

Pour la séance notée, le rapport final devra prendre la forme d'un **notebook Jupyter exécuté**, contenant les réponses, le code Python, les résultats numériques et les graphiques, puis **exporté en HTML ou en PDF**.

— Dans Jupyter, exécuter toutes les cellules, enregistrer le notebook, puis choisir *Exporter / Save and Export Notebook As* pour produire un fichier **HTML** ou **PDF** (selon l'interface et les outils disponibles).

— Il est également possible d'utiliser le terminal après avoir enregistré le notebook :

```
jupyter nbconvert --to html rapport.ipynb
jupyter nbconvert --to pdf rapport.ipynb
```

L'export HTML est le plus simple. L'export PDF avec `nbconvert` *peut nécessiter une installation LaTeX ; si elle n'est pas disponible, choisir HTML.* Il est possible de s'entraîner à l'export avant la séance notée, même en utilisant Spyder pour les TP ordinaires.

Documentations officielles : Spyder (installation), Spyder (éditeur), Jupyter (installation), Jupyter (export HTML/PDF).